

Complejidad algorítmica

Clase práctica

Algoritmos y Estructuras de Datos

Departamento de Computación



DEPARTAMENTO
DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

2^{do} cuatrimestre 2026

Menú del día

1 Funciones y conjuntos

2 Análisis de algoritmos

Cotas de complejidad - $\mathcal{O}$, Ω , Θ

Definición

Sea $g : \mathbb{N} \rightarrow \mathbb{R}$. Entonces:

$$\mathcal{O}(g) \stackrel{\text{def}}{=} \{f : \mathbb{N} \rightarrow \mathbb{R} \mid (\exists n_0 \in \mathbb{N}, c \in \mathbb{R}_{>0}) f(n) \leq c \cdot g(n) \quad \forall n \geq n_0\}$$

Definición

Sea $g : \mathbb{N} \rightarrow \mathbb{R}$. Entonces:

$$\Omega(g) \stackrel{\text{def}}{=} \{f : \mathbb{N} \rightarrow \mathbb{R} \mid (\exists n_0 \in \mathbb{N}, c \in \mathbb{R}_{>0}) f(n) \geq c \cdot g(n) \quad \forall n \geq n_0\}$$

Definición

Sea $g : \mathbb{N} \rightarrow \mathbb{R}$. Entonces:

$$\Theta(g) \stackrel{\text{def}}{=} \{f : \mathbb{N} \rightarrow \mathbb{R} \mid f \in \mathcal{O}(g) \wedge f \in \Omega(g)\}$$

Ejercicio

Demostrar que $n^2 + 5n + 3 \in \Omega(n)$, pero $n^2 + 5n + 3 \notin \mathcal{O}(n)$

Ejercicio

$$n^2 + 5n + 3 \in \Omega(n)$$

Queremos ver que:

$$\exists c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N} \text{ tal que: } n^2 + 5n + 3 \geq cn, \quad \forall n \geq n_0$$

Dividiendo ambos lados por n (con $n > 0$):

$$\frac{n^2 + 5n + 3}{n} \geq c$$

$$n + 5 + \frac{3}{n} \geq c, \quad \forall n \geq n_0$$

$n + 5 + \frac{3}{n}$ es una función creciente, con $c = 1$ y $n \geq 1$ se cumple.
 $\implies n^2 + 5n + 3 \in \Omega(n)$

Ejercicio

$$n^2 + 5n + 3 \notin \mathcal{O}(n)$$

Supongamos que:

$$n^2 + 5n + 3 \in \mathcal{O}(n) \Rightarrow \exists c > 0, \exists n_0 \in \mathbb{N} \text{ tal que:}$$

$$n^2 + 5n + 3 \leq cn, \quad \forall n \geq n_0$$

Dividiendo ambos lados por n :

$$n + 5 + \frac{3}{n} \leq c, \quad \forall n \geq n_0$$

Pero $n + 5 + \frac{3}{n} \rightarrow \infty$ cuando $n \rightarrow \infty$, no puede existir c constante que lo cumpla

$$\Rightarrow n^2 + 5n + 3 \notin \mathcal{O}(n)$$

Propiedades – $\diamond$ es “comodín” de $\mathcal{O}, \Omega, \Theta$

1 Toda f cumple $f \in \diamond(f)$.

Reflexiva

2 $f \in \diamond(g) \implies k \cdot f \in \diamond(g)$ (con k cte.)

3 Regla de la suma:

$$f_1 \in \diamond(g) \wedge f_2 \in \diamond(h) \implies f_1 + f_2 \in \diamond(g+h) = \diamond(\max\{g, h\})$$

4 Regla del producto:

$$f_1 \in \diamond(g) \wedge f_2 \in \diamond(h) \implies f_1 \cdot f_2 \in \diamond(g \cdot h)$$

3 y 4 corresponden al **álgebra de órdenes**. Además 4 implica 2.

$$\bullet f \in \diamond(g) \wedge g \in \diamond(h) \implies f \in \diamond(h)$$

Transitiva

$$\bullet f \in \diamond(g) \implies \diamond(f) \subseteq \diamond(g)$$

$$\bullet \diamond(f) = \diamond(g) \iff f \in \diamond(g) \wedge g \in \diamond(f)$$

$$\text{Como } f \in \Theta(g) \implies g \in \Theta(f)$$

Simétrica

$$\bullet \Theta(f) = \Theta(g) \iff f \in \Theta(g)$$

Ejercicio

Demostrar que $n^2 + 5n + 3 \in \Omega(n)$, pero $n^2 + 5n + 3 \notin \mathcal{O}(n)$

Ejemplo con Propiedades

$$n^2 + 5n + 3 \in \Omega(n),$$

Usamos propiedades:

- Sea $f(n) = n^2$, $g(n) = 5n$, $h(n) = 3$.
- Por **reflexividad**, $f \in \Omega(f)$, $g \in \Omega(g)$ y $h \in \Omega(h)$
- Por **propiedades de las constantes**, $5n \in \Omega(n)$ y $3 \in \Omega(1)$
- Por la **regla de la suma**:

$$n^2 + n + 1 \in \Omega(n^2 + n + 1) = \Omega(\max\{n^2, n, 1\}) = \Omega(n^2)$$

Como $\Omega(n^2) \subseteq \Omega(n)$ entonces $n^2 + 5n + 3 \in \Omega(n)$

Ejemplo con Propiedades

$$n^2 + 5n + 3 \notin \mathcal{O}(n),$$

Usamos propiedades:

- Sea $f(n) = n^2$, $g(n) = 5n$, $h(n) = 3$.
- Por la **regla de la suma**:

$$n^2 + n + 1 \in \Omega(n^2 + n + 1) = \Omega(\max\{n^2, n, 1\}) = \Omega(n^2)$$

La regla de la suma nos da la cota inferior más ajustada.

Como $\Omega(n^2) \cap \mathcal{O}(n) = \emptyset$ y $n^2 + 5n + 3 \in \Omega(n^2)$, entonces $n^2 + 5n + 3 \notin \mathcal{O}(n)$.

Ejercicio

Demostrar que:

- $\mathcal{O}(n^2) \cap \Omega(1) \neq \Theta(n)$

Ejercicio

$$\mathcal{O}(n^2) \cap \Omega(1) \neq \Theta(n)$$

Vamos a mostrar que existe al menos una función en la intersección

$$\mathcal{O}(n^2) \cap \Omega(1)$$

que no pertenece a $\Theta(n)$.

- $\mathcal{O}(n^2)$: funciones que no crecen más rápido que n^2
- $\Omega(1)$: funciones que están acotadas inferiormente por la función constante
- $\Theta(n)$: funciones que crecen linealmente

Ejemplo: Con $f(n) = n^2$

- $f \in \mathcal{O}(n^2)$ y $f \in \Omega(1)$,
- $f \notin \Theta(n)$
- por lo que $f \in \mathcal{O}(n^2) \cap \Omega(1)$ pero $f \notin \Theta(n)$. Los conjuntos no son iguales

Ejercicios

Decidir si son verdaderas o falsas y justificar:

- ❶ $2^n \in \mathcal{O}(1)$
- ❷ $2^n \in \mathcal{O}(n!)$
- ❸ $\Omega(n) \subseteq \mathcal{O}(n^2)$
- ❹ $\mathcal{O}(n) \subseteq \Omega(n)$
- ❺ $\forall k \in \mathbb{N}_0$ vale que $\Omega(n^{k+1}) \subseteq \Omega(n^k)$
- ❻ $f \in \Theta(n^{10}) \implies f \in \Omega(n)$

$$2^n \in \mathcal{O}(1)$$

FALSO. Supongamos que:

$$2^n \in \mathcal{O}(1) \Rightarrow \exists c > 0, \exists n_0 \in \mathbb{N} \text{ tal que:}$$

$$2^n \leq c1, \quad \forall n \geq n_0$$

Pero $2^n \rightarrow \infty$ cuando $n \rightarrow \infty$, no puede existir c constante que lo cumpla

$$\Rightarrow 2^n \notin \mathcal{O}(1)$$

$$2^n \in \mathcal{O}(n!)$$

VERDADERO

Queremos ver que:

$$\exists c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N} \text{ tal que: } 2^n \leq cn!, \quad \forall n \geq n_0$$

$2^n = \prod_{i=1}^n 2$ y $n! = \prod_{i=1}^n i$, hay n términos de cada lado

$$2 * 2 * 2 * 2 * \dots * 2 * 2 \leq c * (1 * 2 * 3 * 4 * \dots * n - 1 * n), \quad \forall n \geq n_0$$

Todos los términos de $n!$ son mayor o iguales a los de 2^n excepto por el primero.

Con $c = 2$,

$$2 * 2 * 2 * 2 * \dots * 2 * 2 \leq 2 * 2 * 3 * 4 * \dots * n - 1 * n), \quad \forall n \geq 1$$

Con $c \geq 2$ y $n \geq 1 \implies 2^n \in \mathcal{O}(n!)$

$$\Omega(n) \subseteq \mathcal{O}(n^2)$$

FALSO. Damos un contraejemplo, necesitamos una función que esté en $\Omega(n)$ pero no en $\mathcal{O}(n^2)$

- $\mathcal{O}(n^2)$: funciones que no crecen más rápido que n^2
- $\Omega(n)$: funciones que crecen igual o más rápido que n

Ejemplo: Con $f(n) = n^5$

- $f \in \Omega(n)$
- $f \notin \mathcal{O}(n^2)$
- por lo que $f \in \Omega(n^2)$ pero $f \notin \mathcal{O}(n^2)$. Por lo tanto, $\Omega(n) \not\subseteq \mathcal{O}(n^2)$

$$\mathcal{O}(n) \subseteq \Omega(n)$$

FALSO. Damos un contraejemplo, necesitamos una función que esté en $\mathcal{O}(n)$ pero no en $\Omega(n)$

- $\mathcal{O}(n)$: funciones que no crecen más rápido que n
- $\Omega(n)$: funciones que crecen igual o más rápido que n

Ejemplo: Con $f(n) = 1$

- $f \in \mathcal{O}(n)$
- $f \notin \Omega(n)$
- por lo que $f \in \Omega(n)$ pero $f \notin \mathcal{O}(n)$. Por lo tanto, $\mathcal{O}(n) \not\subseteq \Omega(n)$

$\forall k \in \mathbb{N}_0$ vale que $\Omega(n^{k+1}) \subseteq \Omega(n^k)$

VERDADERO. Vale por definición. Tomando $c = 1$ y $n_0 = 1$:

$$\begin{aligned} f \in \Omega(n^{k+1}) &\implies \forall n \geq n_0, f(n) \geq cn^{k+1} \\ &\implies \forall n \geq n_0, f(n) \geq cn^k \implies f \in \Omega(n^k) \end{aligned}$$

$$f \in \Theta(n^{10}) \implies f \in \Omega(n)$$

VERDADERO. $f \in \Theta(n^{10}) \implies f \in \Omega(n^{10})$.

Luego, por el ejercicio anterior, $f \in \Omega(n^{10}) \subseteq \Omega(n^9) \subseteq \dots \subseteq \Omega(n)$

Propiedad

Dadas las funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}$. Si existe

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \ell \in \mathbb{R}_{\geq 0} \cup \{+\infty\}$$

Entonces:

- $f \in \Theta(g) \iff 0 < \ell < +\infty$
- $f \in \mathcal{O}(g)$ y $f \notin \Omega(g) \iff \ell = 0$
- $f \in \Omega(g)$ y $f \notin \mathcal{O}(g) \iff \ell = +\infty$

Ejercicio

Demostrar que $n^2 + 5n + 3 \in \Omega(n)$, pero $n^2 + 5n + 3 \notin \mathcal{O}(n)$

Ejercicio

$$n^2 + 5n + 3 \in \Omega(n), \quad \text{pero} \quad n^2 + 5n + 3 \notin \mathcal{O}(n)$$

Usando el límite:

$$\lim_{n \rightarrow \infty} \frac{n^2 + 5n + 3}{n} = \lim_{n \rightarrow \infty} \left(n + 5 + \frac{3}{n} \right) = +\infty$$

Conclusión:

- $\ell = +\infty \Rightarrow f \in \Omega(n) \wedge f \notin \mathcal{O}(n)$
- Por lo tanto: $n^2 + 5n + 3 \in \Omega(n)$ pero no en $\mathcal{O}(n)$

Primeros pasos – Análisis de complejidad

Algorithm BUSQUEDA SECUENCIAL($A : \text{arreglo}(\text{nat}), e : \text{nat}$)
 $\rightarrow \text{bool}$

```

1: var  $i : \text{nat}, n : \text{nat}$ 
2:  $n \leftarrow \text{tam}(A)$ 
3:  $i \leftarrow 0$ 
4: mientras  $i < n \wedge A[i] \neq e$  hacer
5:    $i \leftarrow i + 1$ 
6: devolver  $(i < n)$ 

```

- $f_{\text{mejor}}(n)$ y $f_{\text{peor}}(n)$: **cantidad de operaciones** realizadas para un arreglo de tamaño n en mejor y peor caso respectivamente.

Primeros pasos – Análisis del **peor** caso

Algorithm BUSQUEDA SECUENCIAL($A : \text{arreglo}(\text{nat}), e : \text{nat}$)
 $\rightarrow \text{bool}$

```

1: var  $i : \text{nat}, n : \text{nat}$ 
2:  $n \leftarrow \text{tam}(A)$ 
3:  $i \leftarrow 0$  ▷ línea 2-3: 3
4: mientras  $i < n \wedge A[i] \neq e$  hacer ▷ ciclo:  $(n+1) \cdot 4 + 2 \cdot n$ 
5:    $i \leftarrow i + 1$  ▷ 2
6: devolver  $(i < n)$  ▷ 2

```

- $f_{\text{mejor}}(n)$ y $f_{\text{peor}}(n)$: **cantidad de operaciones** realizadas para un arreglo de tamaño n en mejor y peor caso respectivamente.
- $f_{\text{peor}}(n) = 3 + (n+1) 4 + 2n + 2 = 3 + 4 + 2 + (4 + 2) n$
 $= 9 + 6n$

Ejercicio

Elegir la o las opciones correcta:

- Si el algoritmo G es $O(n)$, siendo n el tamaño de la entrada:
 - ① La cantidad de operaciones del peor caso de G es lineal
 - ② La complejidad de G para cualquier caso está acotada superiormente por n
 - ③ El peor caso no puede ser $\Theta(\log(n))$
 - ④ El peor caso es $O(n)$

Ejercicio

- Si el algoritmo G es $O(n)$, siendo n el tamaño de la entrada:
 - 1 La cantidad de operaciones del peor caso de G es lineal FALSO. $O(n)$ no implica que el peor caso sea exactamente lineal, $O(n)$ incluye también funciones de complejidad logarítmica o constante, por ejemplo.
 - 2 La complejidad de G para cualquier caso está acotada superiormente por n VERDADERO.
 - 3 El peor caso no puede ser $\Theta(\log(n))$ FALSO. Si el peor caso tiene cota exacta logarítmica, cumple que está acotado superiormente por n
 - 4 El mejor caso es $O(n)$ VERDADERO. El peor caso (como todos) está acotado superiormente por n

Errores comunes

O NO ES PEOR CASO

Ω NO ES MEJOR CASO

Θ NO ES CASO PROMEDIO

Errores comunes

O NO ES PEOR CASO

Ω NO ES MEJOR CASO

Θ NO ES CASO PROMEDIO

O , Ω y Θ son conjuntos de funciones. Peor caso, mejor caso y caso promedio son instancias del problema. Cada instancia del problema realiza una cantidad $f_{caso}(n)$ de operaciones. $f_{caso}(n) \in \Diamond(f(n))$.

Ejemplo: En BUSQUEDA SECUENCIAL, el mejor caso realiza una cantidad constante de operaciones, $f_{mejor}(n) \in \Theta(1)$. El peor caso realiza una cantidad constante de operaciones, $f_{peor}(n) \in \Theta(n)$.

Errores comunes

NINGÚN CASO PUEDE
FIJAR EL TAMAÑO DE LA
ENTRADA

Errores comunes

NINGÚN CASO PUEDE FIJAR EL TAMAÑO DE LA ENTRADA

El análisis que hacemos es asintótico **en función del tamaño de la entrada**. Justamente queremos medir cómo crece el costo cuando crece la entrada.

Ejemplo: En `BUSQUEDA SECUENCIAL`, el mejor caso no es que el arreglo tenga tamaño 0, es que, sin importar el tamaño, el elemento esté al principio.

Ejercicio: Dar complejidad en mejor y peor caso

Algorithm SUMAPRODUCTOSMENORES $A100(A : \text{arreglo}(\text{nat}))$
 $\rightarrow \text{nat}$

```
1: si  $\text{tam}(A) = 0$  entonces  
2:   devolver 0  
3: var  $i \leftarrow 0$ ,  $\text{suma} \leftarrow 0$   
4: mientras  $i < \text{tam}(A)$  hacer  
5:   var  $j \leftarrow 0$   
6:   mientras  $j < \text{tam}(A)$  hacer  
7:     si  $A[i] \cdot A[j] < 100$  entonces  
8:        $\text{suma} \leftarrow \text{suma} + A[i] \cdot A[j]$   
9:      $j \leftarrow j + 1$   
10:   $i \leftarrow i + 1$   
11: devolver suma
```

Análisis de complejidad:

SUMAPRODUCTOSMENORES A 100

Algorithm SUMAPRODUCTOSMENORES A 100(A)

1: si $\text{tam}(A) = 0$ entonces	$\triangleright \mathcal{O}(1)$
2: devolver 0	$\triangleright \mathcal{O}(1)$
3: var $i \leftarrow 0$, $\text{suma} \leftarrow 0$	$\triangleright \mathcal{O}(1)$
4: mientras $i < \text{tam}(A)$ hacer	$\triangleright \Theta(n)$ iteraciones
5: var $j \leftarrow 0$	$\triangleright \mathcal{O}(1)$
6: mientras $j < \text{tam}(A)$ hacer	$\triangleright \Theta(n)$ iteraciones
7: si $A[i] \cdot A[j] < 100$ entonces	$\triangleright \mathcal{O}(1)$
8: $\text{suma} \leftarrow \text{suma} + A[i] \cdot A[j]$	$\triangleright \mathcal{O}(1)$
9: $j \leftarrow j + 1$	$\triangleright \mathcal{O}(1)$
10: $i \leftarrow i + 1$	$\triangleright \mathcal{O}(1)$
11: devolver suma	$\triangleright \mathcal{O}(1)$

¿Qué influye en la complejidad?

- Dos ciclos anidados que recorren el arreglo de tamaño n
- Dentro del segundo ciclo hay un *if* pero no afecta la complejidad total (si entra al if sólo hace operaciones de costo constante)

Para este algoritmo el peor y el mejor caso tienen la misma complejidad,

Complejidad:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \Theta(1)$$

$$\Rightarrow T(n) = \Theta \left(\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 \right) = \Theta \left(\sum_{i=0}^{n-1} (n-1) \right) = \Theta(n \cdot n) = \boxed{\Theta(n^2)}$$

Ejercicio: Mayor subsecuencia menor a k

Se tiene un arreglo de **números positivos** A y un valor $k \in \mathbb{N}$. Buscamos el máximo valor de la suma de los elementos de cualquier subsecuencia contigua de A , tal que este no supere k .

Ejemplo: Mayor subsecuencia menor a k

Ejemplo con $k = 10$ y $A = [1, 2, 3, 4, 2]$

El valor máximo ≤ 10 es 10 y está en $[1, 2, 3, 4]$

Ejemplo con $k = 7$ y $A = [2, 1, 2, 3, 6]$

El valor máximo ≤ 7 es 6 y está en $[1, 2, 3]$

Ejemplo con $k = 15$ y $A = [5, 1, 3, 5, 5, 10, 2]$

El valor máximo ≤ 15 es 15 y está en $[5, 10]$

Algoritmo: Mayor subsecuencia menor a k

Algorithm MAYORSUBSEQMENORK(A : Arreglo de $\mathbb{N}$, k : $\mathbb{N}$)

```
1:  $n \leftarrow |A|$ 
2:  $left, right \leftarrow 0$ 
3:  $suma, MejorSuma \leftarrow 0$ 
4: mientras  $right < n$  hacer
5:    $suma \leftarrow suma + A[right]$ 
6:   mientras  $suma > k \wedge left \leq right$  hacer
7:      $suma \leftarrow suma - A[left]$ 
8:      $left \leftarrow left + 1$ 
9:    $MejorSuma \leftarrow \max(MejorSuma, suma)$ 
10:  si  $MejorSuma = k$  entonces
11:    devolver  $k$ 
12:   $right \leftarrow right + 1$ 
13: devolver  $MejorSuma$ 
```

Peor caso: Mayor subsecuencia menor a k

Algorithm MAYORSUBSEQMENORK(A : Arreglo de $\mathbb{N}$, k : $\mathbb{N}$)

1: $n \leftarrow A $	▷ $O(1)$
2: $left, right \leftarrow 0$	▷ $O(1)$
3: $suma, MejorSuma \leftarrow 0$	▷ $O(1)$
4: mientras $right < n$ hacer	▷ n veces
5: $suma \leftarrow suma + A[right]$	▷ $O(1)$
6: mientras $suma > k \wedge left \leq right$ hacer	▷ ? veces
7: $suma \leftarrow suma - A[left]$	▷ $O(1)$
8: $left \leftarrow left + 1$	▷ $O(1)$
9: $MejorSuma \leftarrow \max(MejorSuma, suma)$	▷ $O(1)$
10: si $MejorSuma = k$ entonces	▷ $O(1)$
11: devolver k	▷ $O(1)$
12: $right \leftarrow right + 1$	▷ $O(1)$
13: devolver $MejorSuma$	▷ $O(1)$

Peor caso: Mayor subsecuencia menor a k

El peor caso se da cuando tanto *left* como *right* recorren todos los índices de la secuencia y el ciclo mayor se ejecuta n veces.

¿Qué pasa con el ciclo interno?

- No sabemos cuantas veces se ejecuta por cada iteración
- Sí sabemos que **en total** fueron exactamente n veces, pues *left* toma valores de 0 a $n - 1$.
- Las operaciones dentro de cada ciclo son $\Theta(1)$ y entre los dos ciclos ocurren como mucho $2n$ veces.
- Así la complejidad total nos queda de $\Theta(n)$, con $n = |A|$

Moraleja y **error común**: Ciclos anidados **no quiere decir** que la complejidad de uno se multiplique por la del otro.

Ejercicio: Búsqueda binaria

Algorithm BUSQUEDABINARIA(A : Arreglo ordenado de $\mathbb{Z}$, e : $\mathbb{Z}$)

```
1:  $n \leftarrow |A|$ 
2:  $l \leftarrow 0$ 
3:  $r \leftarrow n - 1$ 
4: mientras  $l \neq r$  hacer
5:      $mid \leftarrow (l + r)/2$ 
6:     si  $A[mid] > e$  entonces
7:          $r \leftarrow mid - 1$ 
8:     else
9:          $l \leftarrow mid + 1$ 
10: devolver  $A[l] = e$ 
```

Peor caso: Búsqueda binaria

Algorithm BUSQUEDABINARIA(A : Arreglo ordenado de $\mathbb{Z}$, e : $\mathbb{Z}$)

1: $n \leftarrow A $	$\triangleright O(1)$
2: $l \leftarrow 0$	$\triangleright O(1)$
3: $r \leftarrow n - 1$	$\triangleright O(1)$
4: mientras $l \neq r$ hacer	$\triangleright ? \text{ veces}$
5: $mid \leftarrow (l + r)/2$	$\triangleright O(1)$
6: si $A[mid] > e$ entonces	$\triangleright O(1)$
7: $r \leftarrow mid - 1$	$\triangleright O(1)$
8: else	
9: $l \leftarrow mid + 1$	$\triangleright O(1)$
10: devolver $A[l] = e$	$\triangleright O(1)$

Peor caso: Búsqueda binaria

- Al comenzar los índices abarcan todo el arreglo, $l = 0$ y $r = |A| - 1$
- En cada iteración, descartamos la mitad de la secuencia
- En la i -ésima iteración, nos queda un arreglo de tamaño $\frac{|A|}{2^i}$ por ver
- El ciclo termina cuando $l = r$, osea, cuando queda solo un elemento por ver.

Sea k la cantidad final de iteraciones que realiza el ciclo, vale que $\frac{|A|}{2^k} = 1$. Pero entonces,

$$\frac{|A|}{2^k} \iff |A| = 2^k \iff \log(|A|) = k$$

La complejidad del peor caso del algoritmo entonces es $O(\log(|A|))$.